

PYTHON PROGRAMMING INTERNSHIP REPORT

LIBRARY BOOK INVENTORY MANAGER

INTERNSHIP PROJECT REPORT

SUBMITTED BY

RIYA LAJIKUMAR

BTECH COMPUTER SCIENCE & ENGINEERING

SUBMITTED TO

SYNTECXHUB– PYTHON PROGRAMMING TASK 2

INTERNSHIP AT

SYNTECXHUB

COURSE

PYTHON PROGRAMMING

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to Syntecxhub for providing me with the opportunity to undertake and complete the project titled "**Library Book Inventory Manager.**" This project has been an excellent learning experience that allowed me to apply theoretical programming concepts to a practical software solution. Through this project, I gained valuable knowledge about Python programming, Object-Oriented Programming, file handling, JSON data storage, and software development methodologies.

I would also like to thank my mentors, faculty members, and friends who supported and guided me throughout the project. Their suggestions, encouragement, and constructive feedback helped me improve the quality of the application and overcome various challenges during development. I am grateful to everyone who directly or indirectly contributed to the successful completion of this project.

ABSTRACT

The Library Book Inventory Manager is a Python-based Command Line Interface (CLI) application developed to manage library book records efficiently. Libraries often require systematic methods for maintaining information about books, tracking issued books, and generating inventory reports. Traditional manual methods are time-consuming and prone to errors. This project aims to provide an automated solution that simplifies library management operations.

The application enables users to add books, search books by title or author, issue books, return books, and generate inventory reports. The project utilizes Object-Oriented Programming concepts to create a structured and maintainable system. JSON file storage is used to ensure permanent record keeping and data persistence.

The system improves efficiency, accuracy, and accessibility of library records while demonstrating important software development concepts such as classes, objects, dictionaries, file handling, searching, and report generation. The project serves as an excellent example of how Python programming can be applied to solve real-world management problems.

TABLE OF CONTENTS

CONTENTS

1. ACKNOWLEDGEMENT
2. ABSTRACT
3. TABLE OF CONTENTS
4. INTRODUCTION
5. LITERATURE SURVEY
6. PROBLEM STATEMENT
7. PROJECT OBJECTIVES
8. SCOPE OF THE PROJECT
9. SYSTEM ANALYSIS
10. SYSTEM DESIGN
11. HARDWARE REQUIREMENTS
12. SOFTWARE REQUIREMENTS
13. TECHNOLOGIES USED
14. ALGORITHM
15. FLOWCHART
16. SOURCE CODE
17. SAMPLE OUTPUTS
18. TESTING AND VALIDATION
19. ADVANTAGES
20. LIMITATIONS
21. FUTURE ENHANCEMENTS
22. CONCLUSION

1.INTRODUCTION

Libraries play a crucial role in educational institutions, research organizations, and public learning environments. They provide access to valuable resources such as books, journals, and educational materials. Managing a large collection of books manually can be difficult and time-consuming. Maintaining accurate records of issued and available books becomes increasingly challenging as the size of the library grows.

The Library Book Inventory Manager is designed to automate the process of managing library records. The system allows librarians or administrators to maintain book information efficiently through a menu-driven interface. Users can add new books, search for existing books, issue books to readers, return books, and generate reports showing inventory status.

The application is developed using Python and follows Object-Oriented Programming principles. It demonstrates how software can simplify administrative tasks while improving data accuracy and management efficiency. The project also provides practical experience in developing real-world applications using Python programming.

2.LITERATURE SURVEY

Traditionally, libraries maintained book records using manual registers and paper-based systems. While these methods were simple to implement, they often resulted in data duplication, inaccurate records, and difficulty in retrieving information quickly. As library collections expanded, the limitations of manual systems became increasingly apparent.

With advancements in information technology, computerized library management systems were introduced to improve operational efficiency. Modern systems use databases, cloud storage, and web-based technologies to provide comprehensive solutions for book inventory management. Features such as automated issue tracking, search functionality, and report generation have become standard components of contemporary library systems.

Python has emerged as a popular programming language for developing management systems due to its simplicity, readability, and extensive library support. JSON-based storage provides a lightweight solution for small and medium-sized applications without requiring complex database systems. This project combines Python and JSON technologies to create an effective inventory management solution.

3.PROBLEM STATEMENT

Libraries require efficient methods for maintaining book records and tracking inventory. Manual record management systems are often inefficient, error-prone, and difficult to maintain. Searching for specific books and monitoring issued books can consume significant time and effort.

The absence of an automated inventory management system can result in inaccurate records, duplicate entries, and poor resource utilization. Therefore, there is a need for a computerized solution that simplifies book management, improves record accuracy, and provides easy access to inventory information.

The objective of this project is to develop a Library Book Inventory Manager that automates library operations and provides an organized approach to managing book records through a simple and user-friendly interface.

4.PROJECT OBJECTIVES

The primary objective of the Library Book Inventory Manager is to develop a reliable and efficient system for managing library books and inventory records.

Specific objectives include:

- To develop a menu-driven application using Python.
- To implement Object-Oriented Programming concepts.
- To maintain book records efficiently.
- To provide book search functionality.
- To track issued and returned books.
- To generate inventory reports.
- To ensure permanent data storage using JSON files.
- To improve accuracy and reduce manual effort.
- To demonstrate practical software development techniques.

5.SCOPE OF THE PROJECT

The Library Book Inventory Manager is designed to assist libraries, schools, colleges, and educational institutions in maintaining book records efficiently. The application provides a centralized platform for storing and managing information about books.

The project includes functionalities such as adding books, searching records, issuing books, returning books, and generating reports. It is suitable for small and medium-sized libraries that require a simple inventory management solution. The application can be extended in the future to include advanced features such as member management, fine calculation, and database integration. The project serves both educational and practical purposes by demonstrating software development concepts while providing a useful management tool.

6.SYSTEM ANALYSIS

- a) **Existing System:** Traditional library systems rely heavily on manual record maintenance. Book information is stored in registers or spreadsheets, making updates and searches time-consuming. Tracking issued books manually increases the possibility of errors and inconsistencies.

Limitations of Existing System

- Time-consuming record management.
- Difficult book searching.
- Increased chances of human error.
- Poor inventory tracking.
- Limited reporting capabilities.

- b) **Proposed System:** The proposed Library Book Inventory Manager automates library operations and provides efficient management of book records. The system stores information electronically and enables fast retrieval of data. Inventory reports help users monitor the status of books effectively.

Benefits of Proposed System

- Improved efficiency.
- Faster searching.
- Accurate record management.
- Inventory tracking.
- Permanent storage using JSON files

7. SYSTEM DESIGN

System design is an important phase of software development that defines the structure and functionality of the application. The Library Book Inventory Manager is designed using Object-Oriented Programming principles to ensure modularity, maintainability, and scalability.

The system consists of two primary classes: the **Book** class and the **Library** class. The Book class represents individual book records and contains attributes such as Book ID, Title, Author, and Status. The Library class manages all operations related to books, including adding, searching, issuing, returning, and generating reports.

a) Input Design

The system accepts the following inputs from users:

- Book ID
- Book Title
- Author Name
- Menu Selection
- Search Keywords

These inputs are validated before processing to ensure accuracy and consistency.

b) Process Design

The application processes user requests through the following operations:

- Add Book
- Search Book
- Issue Book
- Return Book
- Display Books
- Generate Reports
- Save Records

c) Output Design

The outputs generated by the system include:

- Book Details
- Search Results
- Issue and Return Status
- Inventory Reports

- Error and Validation Messages

The outputs are displayed in a structured and readable format through the command-line interface.

8.HARDWARE REQUIREMENTS

The Library Book Inventory Manager is a lightweight application and does not require specialized hardware.

a) Minimum Hardware Requirements

- Processor: Intel Core i3
- RAM: 4 GB
- Storage: 500 MB Free Space
- Keyboard and Mouse
- Monitor

b) Recommended Hardware Requirements

- Processor: Intel Core i5 or above
- RAM: 8 GB
- SSD Storage
- High-Resolution Monitor

These specifications ensure smooth operation of the application.

9.SOFTWARE REQUIREMENTS

The software requirements for developing and executing the project are listed below:

a) Operating System

- Windows 10/11
- Linux
- macOS

b) Development Tools

- Visual Studio Code
- Python Interpreter

c) Programming Language

- Python 3.x

d) Libraries and Modules

- JSON Module
- OS Module

e) Version Control

- GitHub

The software tools used are open-source and easily accessible.

10. TECHNOLOGIES USED

Python: Python is the primary programming language used in this project. It is known for its simplicity, readability, and extensive support for software development.

JSON: JSON (JavaScript Object Notation) is used for permanent storage of book records. It provides a lightweight and structured data format.

Object-Oriented Programming (OOP)

The project utilizes OOP concepts such as:

- Classes
- Objects
- Constructors
- Methods
- Encapsulation

These concepts improve software organization and maintainability

Dictionaries and Lists: Python dictionaries and lists are used for efficient storage, retrieval, and manipulation of book information.

Visual Studio Code: Visual Studio Code serves as the Integrated Development Environment (IDE) for coding, testing, and debugging the application.

GitHub: GitHub is used for source code management, version control, and project hosting.

11.ALGORITHM

Library Book Inventory Manager Algorithm

Step 1: Start the application.

Step 2: Load existing book records from the JSON file.

Step 3: Display the main menu options.

Step 4: Read user choice.

Step 5: If Add Book is selected:

- Enter Book ID.
- Enter Title and Author.
- Store the record.
- Save to JSON file.

Step 6: If Search Book is selected:

- Enter title or author.
- Search matching records.
- Display results.

Step 7: If Issue Book is selected:

- Enter Book ID.
- Verify availability.
- Mark as issued.
- Save changes.

Step 8: If Return Book is selected:

- Enter Book ID.
- Update status to available.
- Save changes.

Step 9: If Display Books is selected:

- Show all book records.

Step 10: If Generate Report is selected:

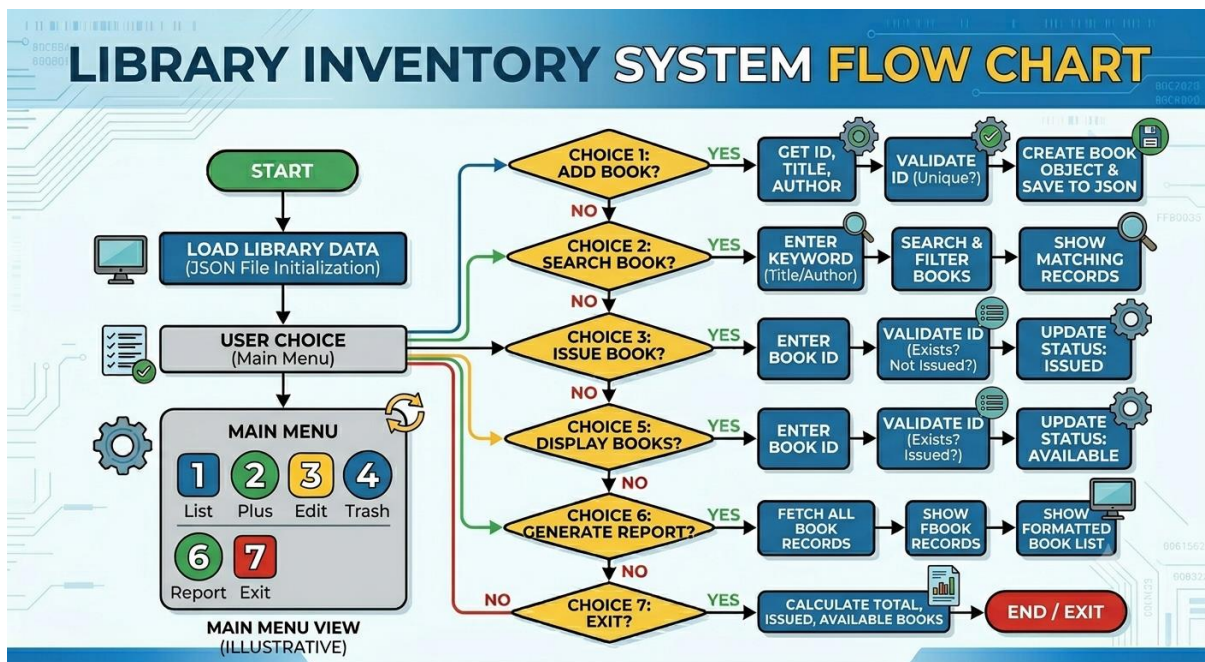
- Display total books.

- Display issued books.
- Display available books.

Step 11: Repeat menu operations until Exit is selected.

Step 12: End the application

12.FLOWCHART



This flowchart visualizes the complete functional logic and structural execution of your Python Library Inventory System script. The program lifecycle begins at the green Start node, which directly initializes the application by loading book data from the local JSON file. Control then moves down to the User Choice panel, which presents the interactive menu containing options for managing book inventory. From the menu, user input splits into a vertical stack of yellow Decision Diamonds that systematically evaluate conditional if-elif-else code branches. Choosing options one through six routes execution horizontally into target CRUD sub-processes like validating unique IDs, parsing keyword text strings, or calculating data summaries. Once a chosen task—such as adding, searching, issuing, or displaying books—concludes its operations, a feedback loop dynamically cycles control back up to the main options display. This continuous logic loop terminates and cleans up the active memory state only when Choice 7 (Exit) is confirmed, successfully concluding at the red End / Exit terminal.

13.SOURCE CODE

The source code of the Library Book Inventory Manager is developed using Python and follows Object-Oriented Programming principles. The code includes:

- Book Class
- Library Class
- JSON File Handling Functions
- Search Functionality
- Issue and Return Functions
- Inventory Report Generation
- Menu-Driven Main Program

Code:

```
import json
```

```
import os
```

```
FILE_NAME = "library_books.json"
```

```
class Book:
```

```
    def __init__(self, book_id, title, author, issued=False):
```

```
        self.book_id = book_id
```

```
        self.title = title
```

```
        self.author = author
```

```
        self.issued = issued
```

```
    def to_dict(self):
```

```
        return {
```

```
            "book_id": self.book_id,
```

```
            "title": self.title,
```

```
            "author": self.author,
```

```
            "issued": self.issued
```

```
}
```

```
class Library:
```

```
    def __init__(self):
```

```
        self.books = {}
```

```
        self.load_books()
```

```
    def load_books(self):
```

```
        if os.path.exists(FILE_NAME):
```

```
            try:
```

```
                with open(FILE_NAME, "r") as file:
```

```
                    data = json.load(file)
```

```
                for book in data:
```

```
                    self.books[book["book_id"]] = Book(
```

```
                        book["book_id"],
```

```
                        book["title"],
```

```
                        book["author"],
```

```
                        book["issued"]
```

```
                    )
```

```
            except:
```

```
                self.books = {}
```

```
    def save_books(self):
```

```
        data = [book.to_dict() for book in self.books.values()]
```

```
        with open(FILE_NAME, "w") as file:
```

```
            json.dump(data, file, indent=4)
```

```
    def add_book(self):
```

```

book_id = input("Enter Book ID: ")

if book_id in self.books:
    print("Book ID already exists!")
    return

title = input("Enter Title: ")
author = input("Enter Author: ")

self.books[book_id] = Book(
    book_id,
    title,
    author
)

self.save_books()
print("Book Added Successfully!")

def search_book(self):
    keyword = input(
        "Enter Title or Author: "
    ).lower()

    found = False

    for book in self.books.values():

        if (keyword in book.title.lower()
            or keyword in book.author.lower()):

```

```

        print("\n-----")
        print("Book ID :", book.book_id)
        print("Title  :", book.title)
        print("Author :", book.author)

        if book.issued:
            print("Status : Issued")
        else:
            print("Status : Available")

        found = True

    if not found:
        print("No matching books found.")

def issue_book(self):
    book_id = input(
        "Enter Book ID to Issue: "
    )

    if book_id not in self.books:
        print("Book not found.")
        return

    if self.books[book_id].issued:
        print("Book already issued.")
        return

    self.books[book_id].issued = True

```



```

self.save_books()

print("Book Issued Successfully!")

def return_book(self):
    book_id = input(
        "Enter Book ID to Return: "
    )

    if book_id not in self.books:
        print("Book not found.")
        return

    if not self.books[book_id].issued:
        print("Book already available.")
        return

    self.books[book_id].issued = False

    self.save_books()

    print("Book Returned Successfully!")

def display_books(self):

    if not self.books:
        print("No books available.")
        return

    print("\n===== BOOK LIST =====")

```

```

for book in self.books.values():

    status = (
        "Issued"
        if book.issued
        else "Available"
    )

    print(
        f'{book.book_id:<10}'
        f'{book.title:<25}'
        f'{book.author:<20}'
        f'{status}'
    )

def generate_report(self):

    total_books = len(self.books)

    issued_books = sum(
        1 for book in self.books.values()
        if book.issued
    )

    available_books = (
        total_books - issued_books
    )

    print("\n===== LIBRARY REPORT =====")

```

```
print("Total Books   :", total_books)
print("Issued Books   :", issued_books)
print("Available Books :", available_books)
```

```
def main():
```

```
    library = Library()
```

```
    while True:
```

```
        print("\n===== LIBRARY BOOK INVENTORY =====")
        print("1. Add Book")
        print("2. Search Book")
        print("3. Issue Book")
        print("4. Return Book")
        print("5. Display Books")
        print("6. Generate Report")
        print("7. Exit")
```

```
        choice = input(
            "Enter Your Choice: "
        )
```

```
        if choice == "1":
            library.add_book()
```

```
        elif choice == "2":
            library.search_book()
```

```
        elif choice == "3":
```

```
        library.issue_book()

elif choice == "4":
    library.return_book()

elif choice == "5":
    library.display_books()

elif choice == "6":
    library.generate_report()

elif choice == "7":
    print("Thank You!")
    break

else:
    print("Invalid Choice!")

if __name__ == "__main__":
    main()
```

14. SAMPLE OUTPUT

OUTPUT

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 1

Enter Book ID: B101

Enter Title: The House in the Cerulean Sea

Enter Author: TJ Klune

Book Added Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 1

Enter Book ID: B102

Enter Title: A Man Called Ove

Enter Author: Fredrik Backman

Book Added Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 1

Enter Book ID: B103

Enter Title: The Secret Garden

Enter Author: Frances Hodgson Burnett

Book Added Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 1

Enter Book ID: B104

Enter Title: Red, White & Royal Blue

Enter Author: Casey McQuiston

Book Added Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 1

Enter Book ID: B105

Enter Title: Anne of Green Gables

Enter Author: L.M. Montgomery

Book Added Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 5

===== BOOK LIST =====

B101	The House in the Cerulean Sea	TJ Klune	Available
B102	A Man Called Ove	Fredrik Backman	Available
B103	The Secret Garden	Frances Hodgson Burnett	Available
B104	Red, White & Royal Blue	Casey McQuiston	Available

B105 Anne of Green Gables L.M. Montgomery Available

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 2

Enter Title or Author: Anne of Green Gables

Book ID : B105

Title : Anne of Green Gables

Author : L.M. Montgomery

Status : Available

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 3

Enter Book ID to Issue: B103

Book Issued Successfully!

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 4

Enter Book ID to Return: B105

Book already available.

===== LIBRARY BOOK INVENTORY =====

1. Add Book
2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit

Enter Your Choice: 6

===== LIBRARY REPORT =====

Total Books : 5

Issued Books : 1

Available Books : 4

===== LIBRARY BOOK INVENTORY =====

1. Add Book

2. Search Book
3. Issue Book
4. Return Book
5. Display Books
6. Generate Report
7. Exit
Enter Your Choice: 7
Thank You!

15.TESTING AND VALIDATION

Testing ensures that the application performs correctly under various conditions.

Test Case	Expected Result	Status
Add Book	Book Added	Pass
Search Book	Record Found	Pass
Issue Book	Status Updated	Pass
Return Book	Status Updated	Pass
Display Books	Records Displayed	Pass
Generate Report	Correct Statistics	Pass
Invalid Input	Error Message	Pass

All functionalities were tested successfully and produced the expected results.

16.ADVANTAGES

- Easy-to-use interface.
- Fast book searching.
- Efficient inventory management.
- Permanent JSON data storage.
- Tracks issued and available books.
- Generates useful reports.
- Demonstrates OOP concepts.
- Lightweight application.
- Easy maintenance and modification.

17.LIMITATIONS

- Command-line interface only.
- Suitable mainly for small libraries.
- No database connectivity.
- Single-user environment.
- Limited search filters.
- No graphical user interface.

18.FUTURE ENHANCEMENTS

The Library Book Inventory Manager can be enhanced with advanced features such as:

- Graphical User Interface using Tkinter.
- MySQL or SQLite database integration.
- Student/member management.
- Fine calculation system.
- Barcode scanning support.

- Book reservation functionality.
- Email notifications.
- Cloud-based storage.
- Web application deployment.
- Mobile application integration.

These enhancements can significantly increase the usability and scalability of the system.

19.CONCLUSION

The Library Book Inventory Manager successfully provides an efficient and organized solution for managing library book records. The application automates important tasks such as adding books, searching records, issuing books, returning books, and generating inventory reports. By replacing manual record-keeping methods, the system improves efficiency, accuracy, and accessibility of library information.

The project demonstrates the practical implementation of Python programming concepts, including Object-Oriented Programming, file handling, JSON storage, searching techniques, and report generation. The use of classes and dictionaries provides a structured approach to managing book data, while JSON storage ensures permanent record maintenance.

Throughout the development process, valuable experience was gained in software design, coding, testing, and debugging. The project strengthened understanding of programming fundamentals and highlighted the importance of organized data management systems. Testing results confirmed that the application performs reliably and meets all specified objectives.

In conclusion, the Library Book Inventory Manager is a practical, user-friendly, and educational software application that effectively manages library inventories. It serves as an excellent example of applying theoretical programming concepts to solve real-world problems and provides a strong foundation for developing more advanced library management systems in the future.